

Game Engine Concepts

Fachhochschule Oberösterreich

Agenda

-
1. Overview & Motivation
 2. Game Engines
 3. Game Loop & Render Pipeline
 4. Graphics, Animation, Physics
 5. Human Interface Devices
 6. Game Scripting



about me

Dr. Andreas Riegler



Introduction: Overview & Motivation

Overview

The Game Developer's Mind

- A multi-level architecture
- Users only see the tip of the iceberg



Overview

The Game Developer's Mind

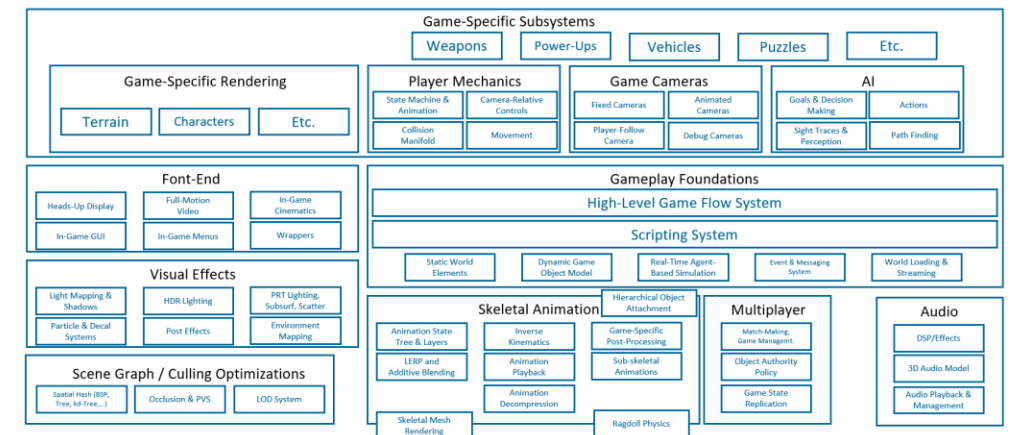
- Fully rendered 3D game scene with interactive components
- Beneath lies a 3D game engine



Overview

The Game Developer's Mind

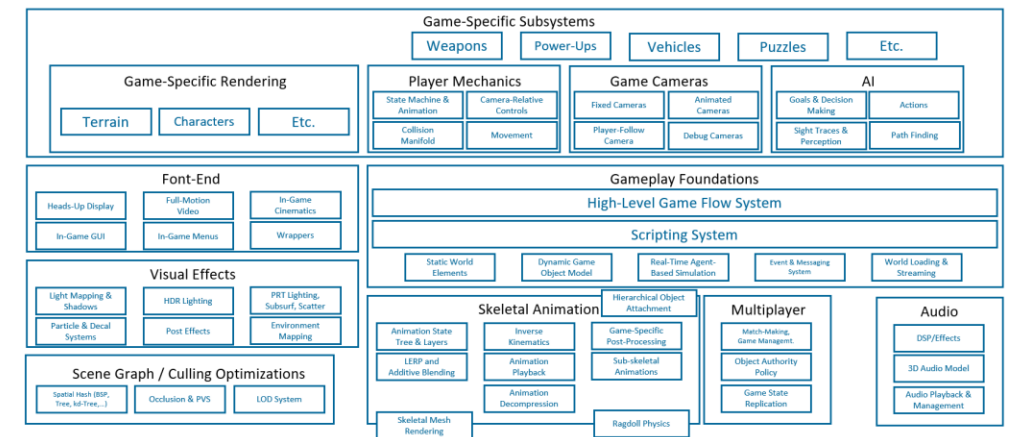
- The game engine is a software framework that separates between core components
- Graphics, game mechanics, physics/collisions, front-end, visual effects, scene graphs, etc.
- Components communicate and act based on code



Overview

The Game Developer's Mind

- The code is nothing else than subsequent instructions in a given programming language
- For a computer scientist, the language doesn't matter, because the concepts stay the same

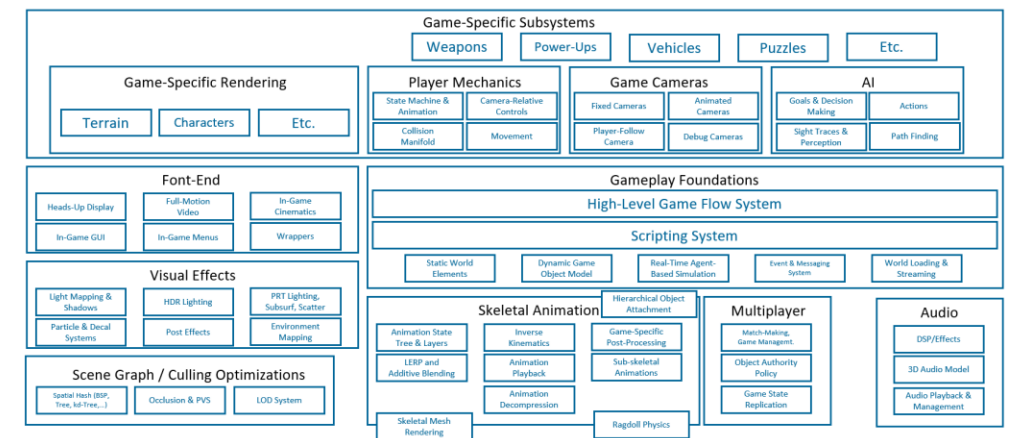


```
// Called to bind functionality to input
void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}
```


Overview

The Game Developer's Mind

- The programs themselves are translated into machine code
- The machine code is the atomic language of a particular computer
- Computers are both complex and entirely simple



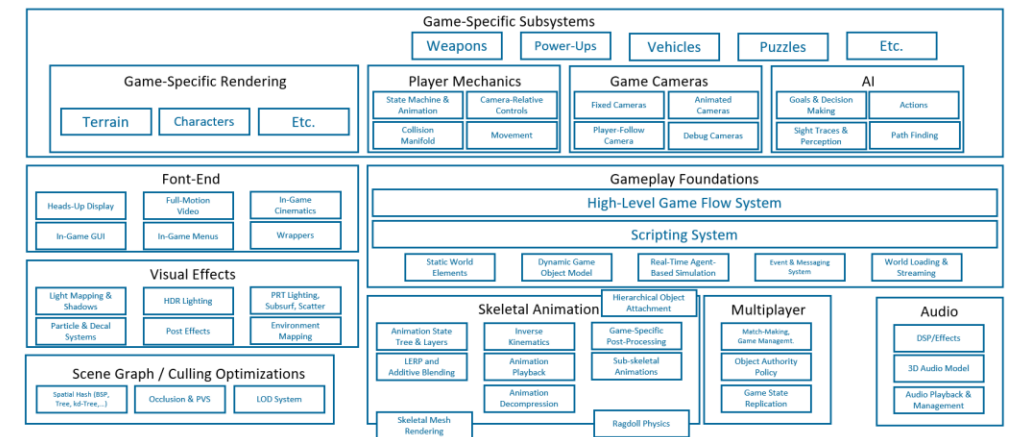
```
// Called to bind functionality to input
void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}

00000000: 65 6C 6C 6F 57 6F 72 6C:64 01 00 10 6A 61 76 61 : elloWorld@ ▶java
000000C0: 2F 6C 61 6E 67 2F 4F 62:6A 65 63 74 01 00 10 6A : /lang/Object@ ▶j
000000D0: 61 76 61 2F 6C 61 6E 67:2F 53 79 73 74 65 6D 01 : ava/lang/System@
000000E0: 00 03 6F 75 74 01 00 15:4C 6A 61 76 61 2F 69 6F : vout@ SLjava/io
000000F0: 2F 50 72 69 6E 74 53 74:72 65 61 6D 3B 01 00 13 : /PrintStream@ !!
00000100: 6A 61 76 61 2F 69 6F 2F:50 72 69 6E 74 53 74 72 : java/io/PrintStr
00000110: 65 61 6D 01 00 07 70 72:69 6E 74 6C 6E 01 00 15 : ean@ *println@ $
```

Overview

The Game Developer's Mind

- The programs themselves are translated into machine code
- The machine code is the atomic language of a particular computer
- Computers are both complex and entirely simple
- **Over the semester, we will learn about these concepts on hand with the Unity and Godot Game Engine, starting from the most atomic principles.**



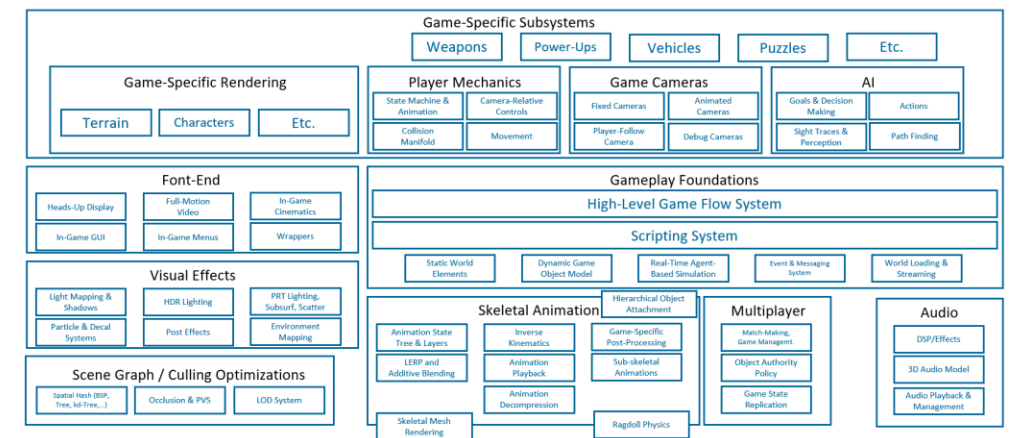
```
// Called to bind functionality to input
void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}

00000000: 65 6C 6C 6F 57 6F 72 6C:64 01 00 10 6A 61 76 61 : elloWorld@ ▶java
000000C0: 2F 6C 61 6E 67 2F 4F 62:6A 65 63 74 01 00 10 6A : /lang/Object@ ▶j
000000D0: 61 76 61 2F 6C 61 6E 67:2F 53 79 73 74 65 6D 01 : ava/lang/System@
000000E0: 00 03 6F 75 74 01 00 15:4C 6A 61 76 61 2F 69 6F : vout@ SLjava/io
000000F0: 2F 50 72 69 6E 74 53 74:72 65 61 6D 3B 01 00 13 : /PrintStream;@ !!
00000100: 6A 61 76 61 2F 69 6F 2F:50 72 69 6E 74 53 74 72 : java/io/PrintStr
00000110: 65 61 6D 01 00 07 70 72:69 6E 74 6C 6E 01 00 15 : ean@ *println@ $
```

Overview

The Game Developer's Mind

- The programs themselves are translated into machine code
- The machine code is the atomic language of a particular computer
- Computers are both complex and entirely simple
- **However, the focus of this first-semester lecture is not the programming aspect, but rather the concepts needed to use game engines for creating games and simulations, etc.**



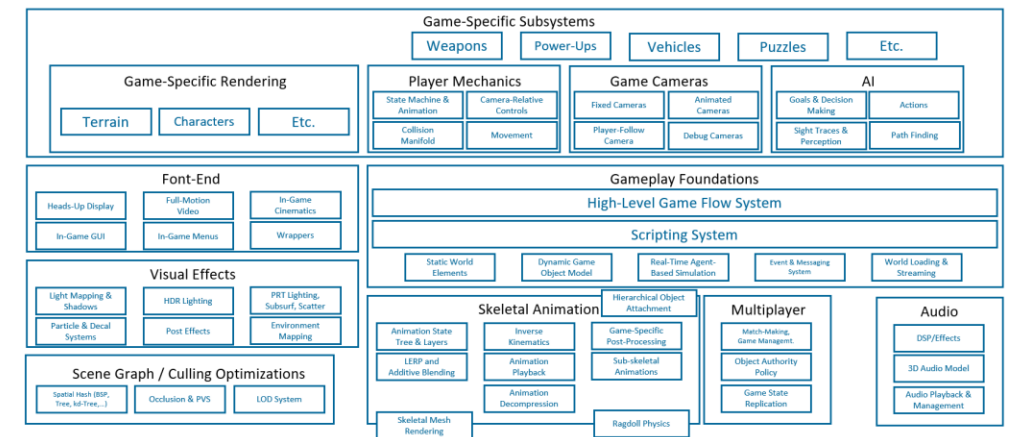
```
// Called to bind functionality to input
void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}

00000000: 65 6C 6C 6F 57 6F 72 6C164 01 00 10 6A 61 76 61 : elloWorld@ java
000000C0: 2F 6C 61 6E 67 2F 4F 6216A 65 63 74 01 00 10 6A : /lang/Object@ j
000000D0: 61 76 61 2F 6C 61 6E 6712F 53 79 73 74 65 6D 01 : ava/lang/System@
000000E0: 00 03 6F 75 74 01 00 1514C 6A 61 76 61 2F 69 6F : vout@ SLjava/io
000000F0: 2F 50 72 69 6E 74 53 74172 65 61 6D 3B 01 00 13 : /PrintStream@ !!
00000100: 6A 61 76 61 2F 69 6F 2F150 72 69 6E 74 53 74 72 : java/io/PrintStr
00000110: 65 61 6D 01 00 07 70 72169 6E 74 6C 6E 01 00 15 : ean@ *println@ $
```


Overview

The Game Developer's Mind

- The programs themselves are translated into machine code
- The machine code is the atomic language of a particular computer
- Computers are both complex and entirely simple
- **Today, we will learn the foundations about game engines**



```
// Called to bind functionality to input
void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}

00000000: 65 6C 6C 6F 57 6F 72 6C 16 01 00 10 6A 61 76 61 : elloWorld@ java
0000000C: 2F 6C 61 6E 67 2F 4F 62 16 65 63 74 01 00 10 6A : /lang/Object@ j
000000D0: 61 76 61 2F 6C 61 6E 67 12F 53 79 73 74 65 6D 01 : ava/lang/System@
000000E0: 00 03 6F 75 74 01 00 15 14C 6A 61 76 61 2F 69 6F : out@ java/io
000000F0: 2F 50 72 69 6E 74 53 74 12 65 61 6D 3B 01 00 13 : /PrintStream@ !!
00000100: 6A 61 76 61 2F 69 6F 2F 50 72 69 6E 74 53 74 72 : java/io/PrintStr
00000110: 65 61 6D 01 00 07 70 72 16 6E 74 6C 6E 01 00 15 : ean@ println@ $
```

Overview

Computing Foundations

“I am too lazy to calculate”*

(Konrad Zuse)

*we don't want to do the computations ourselves, but we still want to understand how they work – for many reasons

Overview

Gottfried Wilhelm Leibniz

- *“It is unworthy to waste the time of outstanding people with menial calculations, because when a machine is used, even the simplest person can write down the results with certainty.”*
- How can we automate calculations?
- “Staffelwalze“ (Leibniz wheel, 1673)
- A simple mechanic allowing to add 2 numbers
- Multiple wheels are connected to allow a carry



Overview

Konrad Zuse

- First binary computer (Z1, 1938)
- Boolean logic and floating point numbers
- 8 instructions, between 1 and 21 cycles per instruction
- 16 word floating point memory
- Destroyed in WWII, but rebuilt by Zuse himself at the German Technikumuseum in Berlin
- First fully digital computer (Z3, 1941)

Overview

Alan Turing

- Cracking the enigma is nice, but not his greatest achievement
- Turing Machine: A model of an abstract computer
- Inspired by a tape recorder – a tape head reads and modifies symbols on a tape as given by instructions provided by a program
- The concept allowed breakthroughs in computability theory (i.e., “Decision Problem” [Entscheidungsproblem], “Halting Problem” [Halteproblem], etc.)
- A Turing machine can simulate “all computer programs”

Overview Programming

→ A program is just a series of subsequent instructions

→ To be “Turing complete” (and thus, allow any program), a language needs to...

- Have some memory to read from and write to
- The possibility to perform subsequent instructions
- A conditional jump (“if”)

```
100 void APlayerPawn::Tick(float DeltaTime)
101 {
102     Super::Tick(DeltaTime);
103 }
104
105 // Called to bind functionality to input
106
107 void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
108 {
109     Super::SetupPlayerInputComponent(PlayerInputComponent);
110 }
111
112 void APlayerPawn::BeginPlay()
113 {
114     UE_LOG(LogTemp, Warning, TEXT("Player Pawn just started running"));
115 }
116
117 ST_Player* APlayerPawn::GetPlayerByIndex(int index)
118 {
119     if (index == 0)
120         return Player0Ref;
121     else return Player1Ref;
122 }
123
124 ST_Player* APlayerPawn::GetPlayerByColor(ChessColor color)
125 {
126     if (color == ChessColor::White)
127         return Player0Ref;
128     else return Player1Ref;
129 }
130
131 ST_Player* APlayerPawn::GetPlayerByColor(ChessColor color)
132 {
133     if (color == ChessColor::White)
134         return Player0Ref;
135     else return Player1Ref;
136 }
137
138 void APlayerPawn::SetPlayerColor(int playerIndex, ChessColor color)
139 {
140     if (playerIndex == 0)
141         Player0Ref->Color = color;
142     if (playerIndex == 1)
143         Player1Ref->Color = color;
144 }
145
146 void APlayerPawn::MoveUp()
```

Output

Show output from: Debug

UnrealEditor.exe (Win64). Loaded C:\Program Files\Epic Games\UE_5.3\Engine\Binaries\Win64\UnrealEditor.exe. Symbols loaded.

The program '[31676] UnrealEditor.exe' has exited with code 0 (0x0).

Solution Explorer

Search Solution Explorer (Ctrl+U)

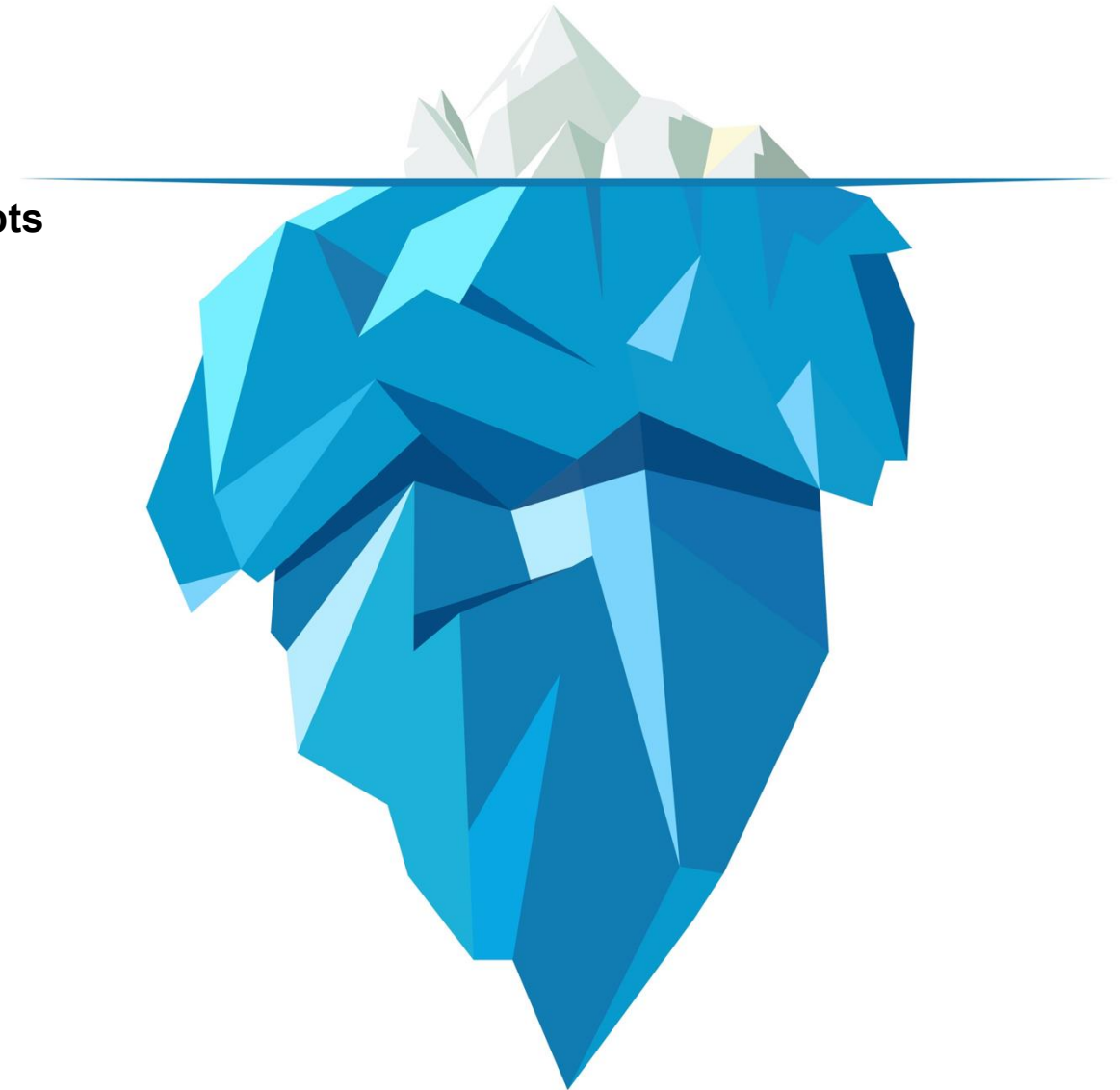
- References
- External Dependencies
- Config
- Source
 - Socchess
 - BaseActor.cpp
 - BaseActor.h
 - BaseCharacter.cpp
 - BaseCharacter.h
 - Board.cpp
 - Board.h
 - BoardSquare.cpp
 - BoardSquare.h
 - ChessDataAsset.cpp
 - ChessDataAsset.h
 - ChessFunctionLibrary.cpp
 - ChessFunctionLibrary.h
 - ChessPiece.cpp
 - ChessPiece.h
 - ChessPlayerController.cpp
 - ChessPlayerController.h
 - LoggingInterface.cpp
 - LoggingInterface.h
 - MyGameMode.cpp
 - MyGameMode.h
 - PlayerPawn.cpp
 - PlayerPawn.h
 - Socchess.Build.cs
 - Socchess.cpp
 - Socchess.h
 - SocchessGameMode.cpp
 - SocchessGameMode.h
 - SocchessGameModeBase.cpp
 - SocchessGameModeBase.h
 - ST_Player.cpp
 - ST_Player.h
 - Socchess.Target.cs
 - SocchessEditor.Target.cs
 - .vsconfig
 - Socchess.uproject
- Programs
- Rules
- Visualizers

Solution Explorer | Git Changes | 18

Overview

The Game Developer's Mind

- A game is “just“ a computer program
- To create a game, we should understand the **underlying concepts and mechanics**



Game Engine Concepts

Course Modalities



Overview

Course Modalities

→ Game Engine Concepts (GEC)

- 12 EH VO, 12 EH UE
- Unity Engine, Godot Game Engine
- Exam: December 16, 2025
- Tutor: Tobias Grantl (MTD 5. Semester)



Game Engine Concepts



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at